

DAY 16 — Primary Keys, Foreign Keys & Constraints

Today we move from "can I query data?" to:

"Can I design a PostgreSQL database that prevents bad data?"

This is a core Data Engineering skill.

You already know joins well, so today we'll connect **relational integrity** → **JOINS** → **production data quality**.

And I'll continue reinforcing your weak area: **LEFT JOIN, NULLs, and parent/child relationships**.

1. Why Constraints Matter

Imagine your RAG system has:

users

documents

document_chunks

embeddings

The relationships are:

users

|

| 1

|

└──< documents

|

| 1

|

└──< document_chunks

|

| 1

|

└──< embeddings

Suppose someone inserts:

`document.user_id = 999999`

but user 999999 doesn't exist.

Without database protection, you have an **orphan document**.

Later:

```
SELECT *
```

```
FROM users u
JOIN documents d
ON u.id = d.user_id;
```

that document simply disappears from the result.

The data was already corrupted.

Constraints prevent this.

2. Primary Key

A **primary key** uniquely identifies a row.

Example:

```
CREATE TABLE users (
id BIGINT PRIMARY KEY,
name TEXT,
email TEXT
);
```

This means:

id

must be:

- unique
- non-NULL

So this is invalid:

```
INSERT INTO users (id, name)
VALUES (1, 'Aarav');
```

```
INSERT INTO users (id, name)
VALUES (1, 'Emma');
```

because:

1

already exists.

3. PRIMARY KEY = UNIQUE + NOT NULL

Conceptually:

id BIGINT PRIMARY KEY

behaves like:

id BIGINT

NOT NULL

UNIQUE

4. Why Primary Keys Matter for Data Engineering

Suppose you ingest documents:

document_id

101

102

103

If document IDs aren't unique, this becomes dangerous:

document_id | title

-----+-----

101 | RAG Guide

101 | PostgreSQL Guide

Now:

```
JOIN documents d
```

```
ON d.id = ...
```

can unexpectedly multiply rows.

So:

Primary keys protect the assumptions your queries make.

5. Composite Primary Key

A primary key can contain multiple columns.

Example:

```
CREATE TABLE document_chunks (
```

```
document_id BIGINT,
```

```
chunk_number INTEGER,
```

```
PRIMARY KEY (
```

```
document_id,
```

```
chunk_number
```

```
)
```

```
);
```

This means:

(document_id, chunk_number)

must be unique.

Therefore:

document_id | chunk_number

-----+-----

101 | 1

101 | 2

101 | 3

102 | 1

is valid.

But:

101 | 1

101 | 1

is not.

This is useful when chunk_number is only unique **within a document**.

6. Foreign Key

A foreign key establishes a relationship between tables.

Example:

```
CREATE TABLE documents (  
  id BIGINT PRIMARY KEY,  
  user_id BIGINT REFERENCES users(id),  
  title TEXT  
);
```

Now:

documents.user_id

must reference an existing:

users.id

assuming the default foreign-key behavior and a referenced key that exists.

7. Parent and Child

Think:

users

↓

parent

documents

↓

child

Foreign key:

documents.user_id

↓

users.id

This is a **referential integrity constraint**.

8. Orphan Records

Without a foreign key:

users

id

--

1

2

3

could coexist with:

documents

id | user_id

---+-----

101 | 1

102 | 999

Document 102 is orphaned.

With:

FOREIGN KEY (user_id)

REFERENCES users(id)

PostgreSQL rejects the invalid reference.

9. NOT NULL

Consider:

```
CREATE TABLE users (  
  id BIGINT PRIMARY KEY,  
  email TEXT NOT NULL  
);
```

This is invalid:

```
INSERT INTO users (id, email)  
VALUES (1, NULL);
```

Because:

email cannot be NULL

Use NOT NULL when the application/database requires a value.

10. UNIQUE

Suppose emails must be unique:

```
CREATE TABLE users (  
  id BIGINT PRIMARY KEY,  
  email TEXT UNIQUE  
);
```

Then:

aarav@example.com

aarav@example.com

cannot appear twice.

Important distinction

PRIMARY KEY:

one primary identity

UNIQUE:

another value must also be unique

A table can have:

1 PRIMARY KEY

multiple UNIQUE constraints

11. UNIQUE + NULL

Here's an important edge case.

Consider:

```
CREATE TABLE users (  
  id BIGINT PRIMARY KEY,  
  email TEXT UNIQUE  
);
```

email is unique, but it can still be NULL unless you explicitly add NOT NULL.

Therefore:

id | email

---+-----

1 | aarav@example.com

2 | NULL

3 | NULL

can be valid.

This surprises people in interviews.

Why?

Because NULL represents an unknown/missing value and isn't treated as an ordinary equal value for uniqueness purposes.

If every user must have an email:

email TEXT NOT NULL UNIQUE

12. CHECK Constraint

You can enforce business rules.

Example:

```
CREATE TABLE documents (  
  id BIGINT PRIMARY KEY,  
  page_count INTEGER,  
  CHECK (page_count > 0)  
);
```

This is invalid:

```
INSERT INTO documents  
VALUES (101, 0);
```

And:

page_count = -10

is also invalid.

13. CHECK for Status

Suppose:

processed

processing

failed

are the only valid statuses.

You could write:

```
CREATE TABLE documents (  
  id BIGINT PRIMARY KEY,  
  status TEXT NOT NULL,  
  CHECK (  
    status IN (  
      'processed',  
      'processing',
```

'failed'

)

)

);

Now:

status = 'banana'

is rejected.

This prevents garbage states.

14. DEFAULT

You can automatically assign values.

Example:

```
CREATE TABLE documents (  
  id BIGINT PRIMARY KEY,  
  status TEXT NOT NULL DEFAULT 'processing'  
);
```

Then:

```
INSERT INTO documents (id)  
VALUES (101);
```

automatically gives:

status = processing

15. Timestamps

A common production pattern:

```
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
```

Example:

```
CREATE TABLE documents (  
  id BIGINT PRIMARY KEY,  
  title TEXT NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

You don't need to manually supply the creation time.

16. Full RAG Example

Let's design:

users

documents

document_chunks

embeddings

Users

```
CREATE TABLE users (  
  id BIGINT PRIMARY KEY,  
  name TEXT NOT NULL,  
  email TEXT NOT NULL UNIQUE,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

Documents

```
CREATE TABLE documents (  
  id BIGINT PRIMARY KEY,  
  
  user_id BIGINT NOT NULL,  
  
  title TEXT NOT NULL,  
  
  page_count INTEGER NOT NULL  
    CHECK (page_count > 0),  
  
  status TEXT NOT NULL  
    CHECK (  
      status IN (  
        'processed',  
        'processing',  
        'failed'  
      )  
    ),  
  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  
  FOREIGN KEY (user_id)  
    REFERENCES users(id)  
);
```

Notice how many protections exist:

id

↓

PRIMARY KEY

user_id

↓

NOT NULL + FOREIGN KEY

title

↓

NOT NULL

page_count

↓

NOT NULL + CHECK

status

↓

NOT NULL + CHECK

created_at

↓

NOT NULL + DEFAULT

This is database engineering.

17. Document Chunks

A strong design:

```
CREATE TABLE document_chunks (  
  document_id BIGINT NOT NULL,
```

```
  chunk_number INTEGER NOT NULL
```

```
  CHECK (chunk_number > 0),
```

```
  content TEXT NOT NULL,
```

```
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
```

```
  PRIMARY KEY (  
    document_id,  
    chunk_number  
  ),
```

```
FOREIGN KEY (document_id)
REFERENCES documents(id)
);
```

Now:

document_id | chunk_number

-----+-----

101 | 1

101 | 2

101 | 3

is valid.

But:

101 | 2

101 | 2

is rejected.

18. Why Composite Keys Matter for RAG

Suppose every document starts chunk numbering from 1.

Then:

Document 101:

1

2

3

Document 102:

1

2

3

chunk_number alone is **not globally unique**.

But:

(document_id, chunk_number)

is unique.

This is a classic relational modeling problem.

19. Foreign Keys and LEFT JOIN

Now connect this to your Day 15 weakness.

If:

documents.user_id

has a foreign key to:

users.id

then every valid document has a parent user.

But that does **not** mean every user has a document.

You can have:

users

1 Aarav

2 Emma

3 Liam

and:

documents

101 → 1

102 → 1

103 → 2

Liam is still a valid user.

Therefore:

```
SELECT
u.id,
COUNT(d.id)
FROM users u
LEFT JOIN documents d
ON u.id = d.user_id
GROUP BY u.id;
```

can correctly return:

1 → 2

2 → 1

3 → 0

Foreign keys enforce:

every child points to a valid parent.

They do **not** enforce:

every parent must have a child.

This distinction is very important.

20. ON DELETE Behavior

Now suppose:

User 1



Document 101

What happens if you delete User 1?

PostgreSQL needs to know what to do with the document.

Foreign keys can specify behavior.

RESTRICT / NO ACTION

Default behavior generally prevents deleting the parent while dependent rows exist.

Conceptually:

DELETE user



dependent documents exist



ERROR

21. ON DELETE CASCADE

You can specify:

FOREIGN KEY (user_id)

REFERENCES users(id)

ON DELETE CASCADE

Then:

DELETE user



documents deleted automatically



chunks?

If chunks also cascade:

User



Documents



Chunks



Embeddings

deleting the user can cascade through the hierarchy.

Production warning

CASCADE is powerful.

A careless:

```
DELETE FROM users
WHERE id = 1;
```

could remove a large amount of dependent data.

Therefore, cascading deletes should be intentional.

22. ON DELETE SET NULL

Another option:

```
FOREIGN KEY (user_id)
REFERENCES users(id)
ON DELETE SET NULL
```

Then:

User deleted

↓

document remains

↓

document.user_id = NULL

But this requires:

user_id

to allow NULL.

So this combination is contradictory:

user_id BIGINT NOT NULL

and:

ON DELETE SET NULL

because PostgreSQL cannot set a NOT NULL column to NULL.

23. Referential Actions

Know these conceptually:

NO ACTION

RESTRICT

CASCADE

SET NULL

SET DEFAULT

You don't need to memorize obscure differences yet.

For interviews, understand the business meaning.

24. Constraint Naming

Production databases should often use explicit names.

Instead of:

```
email TEXT UNIQUE
```

you might define:

```
CONSTRAINT users_email_unique  
UNIQUE (email)
```

And:

```
CONSTRAINT documents_user_fk  
FOREIGN KEY (user_id)  
REFERENCES users(id)
```

This makes database errors easier to understand.

25. ALTER TABLE

Real databases evolve.

Suppose you already have:

```
CREATE TABLE users (  
id BIGINT PRIMARY KEY,  
name TEXT  
);
```

Later:

```
ALTER TABLE users  
ADD COLUMN email TEXT;
```

Add a constraint:

```
ALTER TABLE users  
ADD CONSTRAINT users_email_unique  
UNIQUE (email);
```

Add a foreign key:

```
ALTER TABLE documents  
ADD CONSTRAINT documents_user_fk  
FOREIGN KEY (user_id)  
REFERENCES users(id);
```

This is how production schemas evolve.

26. Constraint Failure Is Good

Suppose:

```
INSERT INTO documents (
```

```
id,  
user_id,  
title,  
page_count,  
status  
)  
VALUES (  
200,  
9999,  
'RAG Guide',  
20,  
'processed'  
);
```

If user 9999 doesn't exist:

PostgreSQL

↓

FOREIGN KEY violation

↓

INSERT rejected

This is not a database failure.

It is **data-quality protection working correctly**.

27. Constraints vs Application Validation

A common interview question.

You might validate:

if `page_count <= 0`: ...

in your FastAPI application.

But should you also have:

```
CHECK (page_count > 0)
```

?

Usually, database-level integrity rules are valuable because data may enter through:

FastAPI

ETL pipeline

batch job

SQL script

admin tool

migration

another service

The database is the final enforcement boundary.

Think:

Application validation

+

Database constraints

=

stronger data integrity

DAY 16 DIFFICULT PRACTICE

Do these yourself.

Challenge 1 — Design a Users Table

Create:

users

Requirements:

- id → primary key
- name → required
- email → required + unique
- country → optional
- account_status → required
- allowed statuses:
 - o active
 - o inactive
 - o suspended
- created_at → automatically populated

Write the complete CREATE TABLE.

Challenge 2 — Document Table

Create:

documents

Requirements:

id

user_id

title

page_count

status

created_at

Rules:

- id primary key
 - user_id required
 - user_id references users
 - title required
 - page_count required
 - page_count > 0
 - status required
 - status must be:
 - o processed
 - o processing
 - o failed
 - created_at automatically populated
-

Challenge 3 — Composite Primary Key

Create:

document_chunks

Requirements:

document_id

chunk_number

content

metadata

created_at

Rules:

- (document_id, chunk_number) is the primary key
- document must exist
- chunk number > 0
- content cannot be NULL
- metadata should support JSON

Choose the PostgreSQL data types.

Challenge 4 — Predict the Result

Assume:

users

id

--

1

2

3

documents

id | user_id

---+-----

101 | 1

102 | 1

103 | 2

Query:

```
SELECT
u.id,
COUNT(*) AS a,
COUNT(d.id) AS b
FROM users u
LEFT JOIN documents d
ON u.id = d.user_id
GROUP BY u.id
ORDER BY u.id;
```

Without executing it, give:

user_id | a | b

-----+---+---

?

for all three users.

Then explain **why**.

Challenge 5 — LEFT JOIN + Constraint Reasoning

Suppose:

documents.user_id

has:

```
FOREIGN KEY (user_id)
REFERENCES users(id)
```

A developer says:

"Because there is a foreign key, every user will have at least one document, so we can use INNER JOIN."

Is that statement correct?

Explain precisely.

Challenge 6 — ON DELETE

You have:

users

↓

documents

↓

document_chunks

Business requirement:

When a user is permanently deleted, all their documents and document chunks should also be deleted automatically.

Design the relevant foreign-key relationships using:

ON DELETE CASCADE

Write the important portions of the table definitions.

DEBUGGING

Challenge 7

This table definition fails to represent the intended business rule:

```
CREATE TABLE documents (  
  id BIGINT PRIMARY KEY,  
  user_id BIGINT NOT NULL,  
  page_count INTEGER NOT NULL,  
  FOREIGN KEY (user_id)  
    REFERENCES users(id)  
  ON DELETE SET NULL  
);
```

What is the problem?

Don't just say "syntax."

Explain the **constraint semantics**.

Challenge 8 — Duplicate Data

Suppose:

```
CREATE TABLE users (  
  id BIGINT PRIMARY KEY,  
  email TEXT UNIQUE  
);
```

The application allows:

user 1 → NULL

user 2 → NULL

Is this necessarily a PostgreSQL uniqueness violation?

Explain.

Then answer:

What would you change if every user must have an email?

INTERVIEW ROUND

Answer these as if you're interviewing for a Data Engineer position.

Q1

What is a primary key?

Q2

What is a foreign key?

Q3

Can a table have multiple foreign keys?

Q4

Can a table have multiple UNIQUE constraints?

Q5

What is the difference between:

PRIMARY KEY

UNIQUE

NOT NULL

CHECK

FOREIGN KEY

Q6

Why doesn't a foreign key guarantee that every parent has a child?

Q7

When would you use a composite primary key?

Q8

Explain:

ON DELETE CASCADE

ON DELETE SET NULL

Q9

Why might database constraints still be necessary if your application already validates the data?

EXPLANATION CHALLENGE

Explain this schema as if you're reviewing it during a production code review:

CREATE TABLE documents (

id BIGINT PRIMARY KEY,

user_id BIGINT NOT NULL,

title TEXT NOT NULL,

page_count INTEGER NOT NULL

CHECK (page_count > 0),

status TEXT NOT NULL

CHECK (

status IN (

'processed',

'processing',

'failed'

)

),

created_at TIMESTAMPTZ NOT NULL

DEFAULT NOW(),

CONSTRAINT documents_user_fk

FOREIGN KEY (user_id)

REFERENCES users(id)

);

Explain:

1. What each constraint protects.
2. Why user_id is both NOT NULL and a foreign key.
3. Why page_count needs a CHECK.
4. Why status needs a CHECK.
5. Why created_at has a default.
6. What happens if someone inserts a nonexistent user_id.
7. What happens if someone inserts page_count = -5.
8. What happens if someone omits created_at.

Day 16 Mastery Target

By the end of Day 16, I want you to think about a database like this:

DATABASE DESIGN

|

└──────────┘

↓ ↓

DATA STRUCTURE DATA INTEGRITY

|

|

Tables Primary Keys

Data Types Foreign Keys

Relationships NOT NULL

UNIQUE

CHECK

DEFAULT

|

↓

Reliable JOINS

Reliable Analytics

Reliable Pipelines

And specifically:

Parent

|

| FK

↓

Child

does **not** mean:

Every parent must have a child

which is exactly why your understanding of:

LEFT JOIN

remains important.

Your assignment: solve **Challenges 1-8 + the 9 interview questions + the explanation challenge**. I will review them as a production SQL/database-design interview, not just mark syntax.